

第 3 天 Java 基础语法

今日内容介绍

- ◆ 引用数据类型（Scanner、Random）
- ◆ 流程控制语句（if、for、while、dowhile、break、continue）

第1章 引用数据类型

1.1 Scanner 类

我们要学的 Scanner 类是属于引用数据类型，我们先了解下引用数据类型。

- 引用数据类型的使用

与定义基本数据类型变量不同，引用数据类型的变量定义及赋值有一个相对固定的步骤或格式。

数据类型 变量名 = new 数据类型();

每种引用数据类型都有其功能，我们可以调用该类型实例的功能。

变量名.方法名();

- Scanner 类

Scanner 类是引用数据类型的一种，我们可以使用该类来完成用户键盘录入，获取到录入的数据。

Scanner 使用步骤：

导包：import java.util.Scanner;

创建对象实例：Scanner sc = new Scanner(System.in);

调用方法：

int i = sc.nextInt(); 用来接收控制台录入的数字

String s = sc.next(); 用来接收控制台录入的字符串

了解完 Scanner 类，我们编写代码来使用下它：ScannerDemo01.java

```
import java.util.Scanner;

public class ScannerDemo01 {

    public static void main(String[] args) {

        //创建 Scanner 引用类型的变量
        Scanner sc = new Scanner(System.in);

        //获取数字
        System.out.println("请输入一个数字");
        int n = sc.nextInt();
        System.out.println("n 的值为" + n);
    }
}
```

```
//获取字符串
System.out.println("请输入一个字符串");
String str = sc.next();
System.out.println("str 的值为" + str);
}
}
```

运行结果如下图所示。

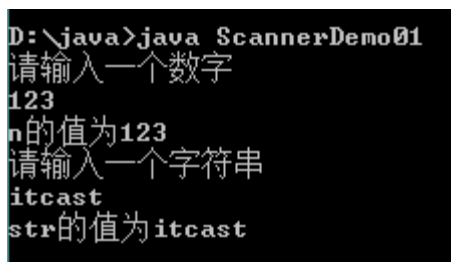


图1-1 运行结果

1.2 随机数类 Random

我们来学习下，用来产生随机数的类 **Random**，它也属于引用数据类型。

这个 **Random** 类，它可以产生多种数据类型的随机数，在这里我们主要介绍生成整数与小数的方式。

- 方法简介

public int nextInt(int maxValue) 产生 [0,maxValue) 范围的随机整数，包含 0，不包含 maxValue;

public double nextDouble() 产生 [0,1) 范围的随机小数，包含 0.0，不包含 1.0。

引用数据类型的使用方式，在学习键盘录入 **Scanner** 时，我们已经学习过，在这里，再次回顾一下：

- **Random** 使用方式：

- **import** 导包：所属包 `java.util.Random`

- 创建实例格式：`Random 变量名 = new Random();`

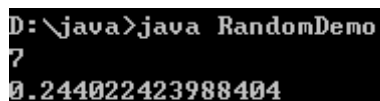
接下来，通过一段代码，一起学习下 **Random** 类的使用，**RandomDemo.java**

```
import java.util.Random;

public class RandomDemo {
    public static void main(String[] args) {
        // 创建 Random 类的实例
        Random r = new Random();
        // 得到 0-100 范围内的随机整数，将产生的随机整数赋值给 i 变量
        int i = r.nextInt(100);
        // 得到 0.0-1.0 范围内的随机小数，将产生的随机小数赋值给 d 变量
        double d = r.nextDouble();
        System.out.println(i);
    }
}
```

```
        System.out.println(d);  
    }  
}
```

运行结果如下图所示：



```
D:\java>java RandomDemo  
0.244022423988404
```

图1-2 运行结果

第2章 流程控制语句

2.1 选择结构 if

接下来要学习的 if 条件语句分为三种语法格式，每一种格式都有它自身的特点，下面我们分别进行介绍。

2.1.1 if 语句

if 语句是指如果满足某种条件，就进行某种处理。例如，小明妈妈跟小明说“如果你考试得了 100 分，星期天就带你去游乐场玩”。这句话可以通过下面的一段伪代码来描述。

```
如果小明考试得了 100 分  
    妈妈星期天带小明去游乐场
```

在上面的伪代码中，“如果”相当于 Java 中的关键字 if，“小明考试得了 100 分”是判断条件，需要用()括起来，“妈妈星期天带小明去游乐场”是执行语句，需要放在{}中。修改后的伪代码如下：

```
if (小明考试得了 100 分) {  
    妈妈星期天带小明去游乐场  
}
```

上面的例子就描述了 if 语句的用法，在 Java 中，if 语句的具体语法格式如下：

```
if (条件语句) {  
    执行语句;  
    .....  
}
```

上述格式中，判断条件是一个布尔值，当判断条件为 true 时，{}中的执行语句才会执行。if 语句的执行流程如下图所示。

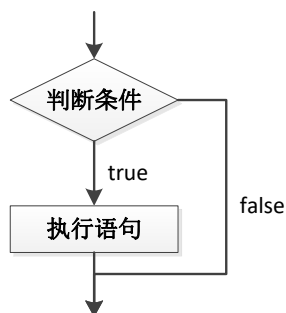


图1-3 if 语句流程图

接下来通过一段代码，学习一下 if 语句的具体用法，IfDemo01.java

```
public class IfDemo01 {  
    public static void main(String[] args) {  
        int x = 5;  
        if (x < 10) {  
            x++;  
        }  
        System.out.println("x=" + x);  
    }  
}
```

运行结果如下图所示。

```
D:\java>java IfDemo01  
x=6
```

图1-4 运行结果

在上述代码中，定义了一个变量 x，其初始值为 5。在 if 语句的判断条件中判断 x 的值是否小于 10，很明显条件成立，{} 中的语句会被执行，变量 x 的值将进行自增。从运行结果可以看出，x 的值已由原来的 5 变成了 6。

2.1.2 if...else 语句

if...else 语句是指如果满足某种条件，就进行某种处理，否则就进行另一种处理。例如，要判断一个正整数的奇偶，如果该数字能被 2 整除则是一个偶数，否则该数字就是一个奇数。if...else 语句具体语法格式如下：

```
if (判断条件) {  
    执行语句 1  
    .....  
} else {  
    执行语句 2  
    .....  
}
```

上述格式中，判断条件是一个布尔值。当判断条件为 true 时，if 后面 {} 中的执行语句 1 会执行。当判断条件为 false 时，else 后面 {} 中的执行语句 2 会执行。if...else 语句的执行流程如下图所示。

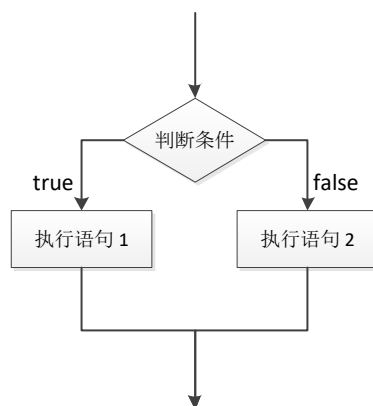


图1-5 if...else 语句流程图

接下来通过一段代码，来实现判断奇偶数的程序，IfDemo02.java

```
public class IfDemo02 {  
    public static void main(String[] args) {  
        int num = 19;  
        if (num % 2 == 0) {  
            // 判断条件成立，num 被 2 整除  
            System.out.println("num 是一个偶数");  
        } else {  
            System.out.println("num 是一个奇数");  
        }  
    }  
}
```

运行结果如下图所示。

```
D:\java>java IfDemo02  
num是一个奇数
```

图1-6 运行结果

上述代码中，变量 num 的值为 19，模以 2 的结果为 1，不等于 0，判断条件不成立。因此会执行 else 后面{}中的语句，打印“num 是一个奇数”。

2.1.3 if...else if...else 语句

if...else if...else 语句用于对多个条件进行判断，进行多种不同的处理。例如，对一个学生的考试成绩进行等级的划分，如果分数大于 80 分等级为优，否则，如果分数大于 70 分等级为良，否则，如果分数大于 60 分等级为中，否则，等级为差。if...else if...else 语句具体语法格式如下：

```
if (判断条件 1) {  
    执行语句 1  
} else if (判断条件 2) {  
    执行语句 2  
}  
...  
else if (判断条件 n) {
```

```
    执行语句 n
} else {
    执行语句 n+1
}
```

上述格式中，判断条件是一个布尔值。当判断条件 1 为 `true` 时，`if` 后面{}中的执行语句 1 会执行。当判断条件 1 为 `false` 时，会继续执行判断条件 2，如果为 `true` 则执行语句 2，以此类推，如果所有的判断条件都为 `false`，则意味着所有条件均未满足，`else` 后面{}中的执行语句 `n+1` 会执行。`if...else if...else` 语句的执行流程如下图所示。

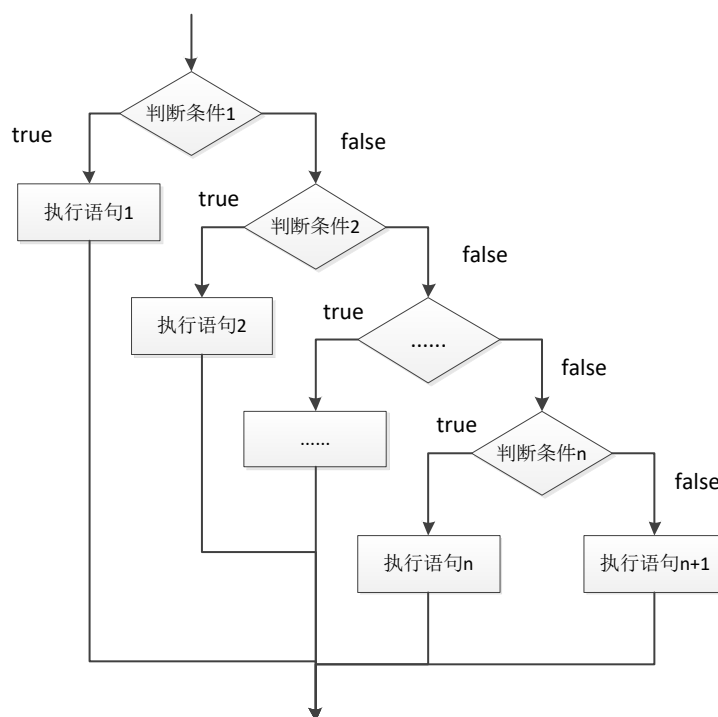


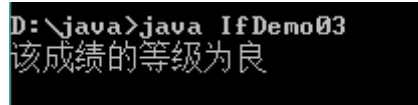
图1-7 `if...else if...else` 语句的流程图

接下来通过一段代码，来实现对学生考试成绩进行等级划分的程序，`IfDemo03.java`

```
public class IfDemo03 {
    public static void main(String[] args) {
        int grade = 75; // 定义学生成绩
        if (grade > 80) {
            // 满足条件 grade > 80
            System.out.println("该成绩的等级为优");
        } else if (grade > 70) {
            // 不满足条件 grade > 80，但满足条件 grade > 70
            System.out.println("该成绩的等级为良");
        } else if (grade > 60) {
            // 不满足条件 grade > 70，但满足条件 grade > 60
            System.out.println("该成绩的等级为中");
        } else {
            // 不满足条件 grade > 60
            System.out.println("该成绩的等级为差");
        }
    }
}
```

```
}  
}
```

运行结果如下图所示。



```
D:\java>java IfDemo03  
该成绩的等级为良
```

图1-8 运行结果

上述代码中，定义了学生成绩 `grade` 为 75。它不满足第一个判断条件 `grade>80`，会执行第二个判断条件 `grade>70`，条件成立，因此会打印“该成绩的等级为良”。

2.1.4 选择结构 if 语句与三元运算转换

三元运算符，它和 `if-else` 语句类似，语法如下：

判断条件 ? 表达式 1 : 表达式 2

三元运算符会得到一个结果，通常用于对某个变量进行赋值，当判断条件成立时，运算结果为表达式 1 的值，否则结果为表达式 2 的值。

例如求两个数 `x`、`y` 中的较大者，如果用 `if...else` 语句来实现，具体代码如下：

```
int x = 0;  
int y = 1;  
int max;  
if (x > y) {  
    max = x;  
} else {  
    max = y;  
}
```

上面的代码运行之后，变量 `max` 的值为 1。其中 3-8 行的代码可以使用下面的三元运算来替换。

```
int max = x > y ? x : y;
```

2.2 循环语句 while

`while` 循环语句和选择结构 `if` 语句有些相似，都是根据条件判断来决定是否执行大括号内的执行语句。区别在于，`while` 语句会反复地进行条件判断，只要条件成立，`{}` 内的执行语句就会执行，直到条件不成立，`while` 循环结束。`while` 循环语句的语法结构如下：

```
while(循环条件) {  
    执行语句  
    .....  
}
```

在上面的语法结构中，`{}` 中的执行语句被称作循环体，循环体是否执行取决于循环条件。当循环条件为 `true` 时，循环体就会执行。循环体执行完毕时会继续判断循环条件，如条件仍为 `true` 则会继续执行，直到循环条件为 `false` 时，整个循环过程才会结束。

`while` 循环的执行流程如下图所示。

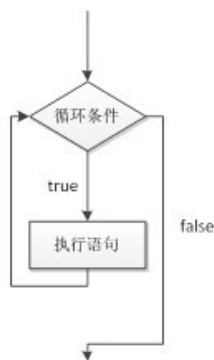


图1-9 while 循环的流程图

接下来通过一段代码，来实现打印 1~4 之间的自然数，WhileDemo.java

```
public class WhileDemo {  
    public static void main(String[] args) {  
        int x = 1; // 定义变量 x，初始值为 1  
        while (x <= 4) { // 循环条件  
            System.out.println("x = " + x); // 条件成立，打印 x 的值  
            x++; // x 进行自增  
        }  
    }  
}
```

运行结果如下图所示。

```
D:\java>java WhileDemo  
x = 1  
x = 2  
x = 3  
x = 4
```

图1-10 运行结果

在上述代码中，x 初始值为 1，在满足循环条件 $x \leq 4$ 的情况下，循环体会重复执行，打印 x 的值并让 x 进行自增。因此打印结果中 x 的值分别为 1、2、3、4。

大家要注意的是，代码 `x++` 用于在每次循环时改变变量 x 的值，从而达到最终改变循环条件的目的。如果没有这行代码，整个循环会进入无限循环的状态，永远不会结束。

2.3 循环语句 for

for 循环语句是最常用的循环语句，一般用在循环次数已知的情况下。for 循环语句的语法格式如下：

```
for (初始化表达式; 循环条件; 操作表达式) {  
    执行语句  
    .....  
}
```

在上面的语法结构中，for 关键字后面()中包括了三部分内容：初始化表达式、循环条件和操作表达式，它们之间用“;”分隔，{}中的执行语句为循环体。

接下来分别用①表示初始化表达式、②表示循环条件、③表示操作表达式、④表示循环体，通

过序号来具体分析 for 循环的执行流程。具体如下：

```
for (① ; ② ; ③) {  
    ④  
}
```

第一步，执行①

第二步，执行②，如果判断结果为 true，执行第三步，如果判断结果为 false，执行第五步

第三步，执行④

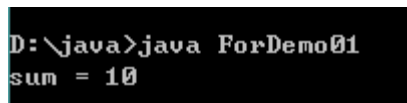
第四步，执行③，然后重复执行第二步

第五步，退出循环

接下来通过一个案例对自然数 1~4 进行求和，如下所示。ForDemo01.java

```
public class ForDemo01 {  
    public static void main(String[] args) {  
        int sum = 0; // 定义变量 sum，用于记住累加的和  
        for (int i = 1; i <= 4; i++) { // i 的值会在 1~4 之间变化  
            sum += i; // 实现 sum 与 i 的累加  
        }  
        System.out.println("sum = " + sum); // 打印累加的和  
    }  
}
```

运行结果如下图所示。



```
D:\java>java ForDemo01  
sum = 10
```

图1-11 运行结果

上述代码中，变量 i 的初始值为 1，在判断条件 i<=4 为 true 的情况下，会执行循环体 sum+=i，执行完毕后，会执行操作表达式 i++，i 的值变为 2，然后继续进行条件判断，开始下一次循环，直到 i=5 时，条件 i<=4 为 false，结束循环，执行 for 循环后面的代码，打印“sum=10”。

为了让初学者能熟悉整个 for 循环的执行过程，现将上述代码运行期间每次循环中变量 sum 和 i 的值通过表 2-11 罗列出来。

表2-1 sum 和 i 循环中的值

循环次数	sum	i
第一次	1	1
第二次	3	2
第三次	6	3
第四次	10	4

2.4 循环语句 do...while

do...while 循环语句和 while 循环语句功能类似，其语法结构如下：

```
do {  
    执行语句  
    .....  
} while(循环条件);
```

在上面的语法结构中，关键字 **do** 后面{}中的执行语句是循环体。**do...while** 循环语句将循环条件放在了循环体的后面。这也就意味着，循环体会无条件执行一次，然后再根据循环条件来决定是否继续执行。

do...while 循环的执行流程如下图所示。

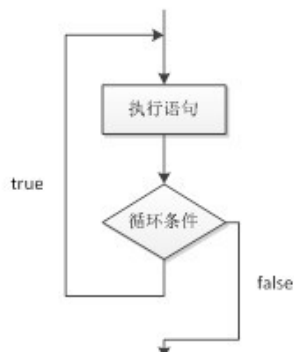


图1-12 do...while 循环的执行流程

接下来使用 **do...while** 循环语句来实现打印 1~4 之间的自然数 **DoWhileDemo.java**。

```
public class DoWhileDemo {  
    public static void main(String[] args) {  
        int x = 1; // 定义变量 x，初始值为 1  
        do {  
            System.out.println("x = " + x); // 打印 x 的值  
            x++; // 将 x 的值自增  
        } while (x <= 4); // 循环条件  
    }  
}
```

运行结果如下图所示。

```
D:\java>java DoWhileDemo  
x = 1  
x = 2  
x = 3  
x = 4
```

图1-13 运行结果

我们发现 **WhileDemo.java** 与 **DoWhileDemo.java** 运行的结果一致，这说明什么呢？这说明 **do...while** 循环和 **while** 循环能实现同样的功能。然而在程序运行过程中，这两种语句还是有差别的。如果循环条件在循环语句开始时就不成立，那么 **while** 循环的循环体一次都不会执行，而 **do...while** 循环的循环体还是会执行一次。若将 **DoWhileDemo.java** 中的循环条件 **x<=4** 改为 **x < 1**，**DoWhileDemo.java** 运行结果会打印 **x=1**，而 **WhileDemo.java** 运行结果什么也不会打印。

2.5 无限循环

最简单无限循环格式：

while(true){}

或

for(;;){}

无限循环存在的原因是因为并不知道循环多少次，而是根据某些条件，来控制循环。

2.6 循环嵌套

嵌套循环是指在一个循环语句的循环体中再定义一个循环语句的语法结构。while、do...while、for 循环语句都可以进行嵌套，并且它们之间也可以互相嵌套，如最常见的在 for 循环中嵌套 for 循环，格式如下：

```
for(初始化表达式; 循环条件; 操作表达式) {  
    .....  
    for(初始化表达式; 循环条件; 操作表达式) {  
        执行语句  
        .....  
    }  
    .....  
}
```

接下来通过一个练习，来实现使用“*”打印直角三角形，如下所示。ForForDemo.java

```
1 public class ForForDemo {  
2     public static void main(String[] args) {  
3         int i, j; // 定义两个循环变量  
4         for (i = 1; i <= 9; i++) { // 外层循环  
5             for (j = 1; j <= i; j++) { // 内层循环  
6                 System.out.print("*"); // 打印*  
7             }  
8             System.out.print("\n"); // 换行  
9         }  
10    }  
11 }
```

运行结果如下所示。

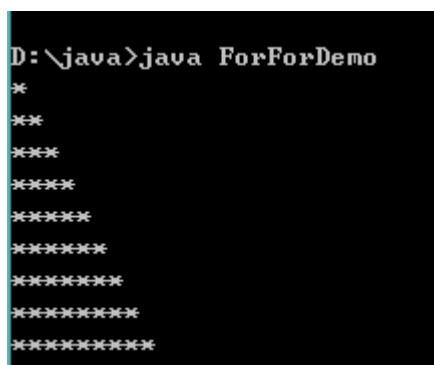


图1-14 运行结果

在上述代码中定义了两层 for 循环，分别为外层循环和内层循环，外层循环用于控制打印的行数，内层循环用于打印“*”，每一行的“*”个数逐行增加，最后输出一个直角三角形。由于嵌套循环程序比较复杂，下面分步骤进行详细地讲解，具体如下：

第一步，在第 3 行代码定义了两个循环变量 i 和 j，其中 i 为外层循环变量，j 为内层循环变量。

第二步，在第 4 行代码将 i 初始化为 1，条件 i <= 9 为 true，首次进入外层循环的循环体。

第三步，在第 5 行代码将 j 初始化为 1，由于此时 i 的值为 1，条件 j <= i 为 true，首次进入内层循环的循环体，打印一个“*”。

第四步，执行第 5 行代码中内层循环的操作表达式 `j++`，将 `j` 的值自增为 2。

第五步，执行第 5 行代码中的判断条件 `j<=i`，判断结果为 `false`，内层循环结束。执行后面的代码，打印换行符。

第六步，执行第 4 行代码中外层循环的操作表达式 `i++`，将 `i` 的值自增为 2。

第七步，执行第 4 行代码中的判断条件 `i<=9`，判断结果为 `true`，进入外层循环的循环体，继续执行内层循环。

第八步，由于 `i` 的值为 2，内层循环会执行两次，即在第 2 行打印两个 “*”。在内层循环结束时打印换行符。

第九步，以此类推，在第 3 行会打印 3 个 “*”，逐行递增，直到 `i` 的值为 10 时，外层循环的判断条件 `i<= 9` 结果为 `false`，外层循环结束，整个程序也就结束了。

2.7 跳转语句（break、continue）

跳转语句用于实现循环执行过程中程序流程的跳转，在 Java 中的跳转语句有 `break` 语句和 `continue` 语句。接下来分别进行详细地讲解。

1、break 语句

在 `switch` 条件语句和循环语句中都可以使用 `break` 语句。当它出现在 `switch` 条件语句中时，作用是终止某个 `case` 并跳出 `switch` 结构。当它出现在循环语句中，作用是跳出循环语句，执行后面的代码。关于在 `switch` 语句中使用 `break` 前面的例程已经用过了，接下来通过下面一段代码，实现将当变量 `x` 的值为 3 时，使用 `break` 语句跳出循环，代码如下所示。BreakDemo.java

```
public class BreakDemo {
    public static void main(String[] args) {
        int x = 1; // 定义变量 x，初始值为 1
        while (x <= 4) { // 循环条件
            System.out.println("x = " + x); // 条件成立，打印 x 的值
            if (x == 3) {
                break;
            }
            x++; // x 进行自增
        }
    }
}
```

运行结果如下图所示。

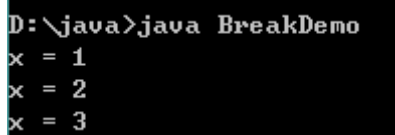


图1-15 运行结果

在上述带代码中，通过 `while` 循环打印 `x` 的值，当 `x` 的值为 3 时使用 `break` 语句跳出循环。因此打印结果中并没有出现 “`x=4`”。

2、标记

当 `break` 语句出现在嵌套循环中的内层循环时，它只能跳出内层循环，如果想使用 `break` 语句跳出外层循环则需要对外层循环添加标记。接下来将 `ForForDemo.java` 稍作修改，控制程序只打印 4

行“*”，如下所示。BreakDemo02.java

```
public class BreakDemo02 {
    public static void main(String[] args) {
        int i, j; // 定义两个循环变量
        itcast: for (i = 1; i <= 9; i++) { // 外层循环
            for (j = 1; j <= i; j++) { // 内层循环
                if (i > 4) { // 判断 i 的值是否大于 4
                    break itcast; // 跳出外层循环
                }
                System.out.print("*"); // 打印*
            }
            System.out.print("\n"); // 换行
        }
    }
}
```

运行结果如下图所示。

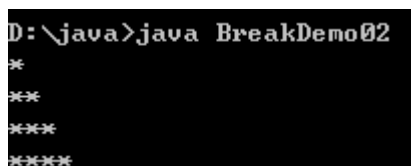


图1-16 运行结果

BreakDemo02.java 与 ForForDemo.java 实现原理类似，只是在外层 for 循环前面增加了标记“itcast”。当 i>4 时，使用 break itcast;语句跳出外层循环。因此程序只打印了 4 行“*”。

3、continue 语句

continue 语句用在循环语句中，它的作用是终止本次循环，执行下一次循环。接下来通过一个练习对 1~100 之内的奇数求和，ContinueDemo.java

```
public class ContinueDemo {
    public static void main(String[] args) {
        int sum = 0; // 定义变量 sum，用于记住和
        for (int i = 1; i <= 100; i++) {
            if (i % 2 == 0) { // i 是一个偶数，不累加
                continue; // 结束本次循环
            }
            sum += i; // 实现 sum 和 i 的累加
        }
        System.out.println("sum = " + sum);
    }
}
```

运行结果如下图所示。



图1-17 运行结果

上述代码中，使用 for 循环让变量 i 的值在 1~100 之间循环，在循环过程中，当 i 的值为偶数时，

将执行 `continue` 语句结束本次循环，进入下一次循环。当 `i` 的值为奇数时，`sum` 和 `i` 进行累加，最终得到 1~100 之间所有奇数的和，打印 “`sum = 2500`”。

在嵌套循环语句中，`continue` 语句后面也可以通过使用标记的方式结束本次外层循环，用法与 `break` 语句相似，在此不再举例说明。

第3章 猜数字案例

3.1 案例介绍

我们在日常生活中，经常与朋友们玩猜数字的游戏，非常的有趣。现在我们通过 `java` 把这个猜数字游戏编写出来。

猜数字案例是要完成什么样的功能呢？顾名思义，这个游戏就是你出个数字、我来猜。

游戏操作如下：

- 后台预先生成一个 1-100 之间的随机数，用户键盘录入猜数字
- 如果猜对了，打印 “恭喜您，答对了”
- 如果猜错了
 - 猜大了：打印 “sorry，您猜大了!”
 - 猜小了：打印 “sorry，您猜小了!”
- 直到数字猜到为止

在案例开始之前，我们先要学习如下知识点：

- 随机数类 `Random`
- 选择结构 `if`
- 循环结构 `while`

3.2 案例需求分析、

猜数字案例的需求如下：

- 后台预先生成一个 1-100 之间的随机数，用户键盘录入猜数字
- 如果猜对了，打印 “恭喜您，答对了”
- 如果猜错了
 - 猜大了：打印 “sorry，您猜大了!”
 - 猜小了：打印 “sorry，您猜小了!”
- 直到数字猜到为止

我们经过分析，得出如下的分析结果：

- 1. 通过 `Random` 类中方法 `nextInt()`，生成一个 1-100 之间的随机数
- 2. 输入猜的数字

- 3. 通过 while 循环，进行猜数字对错判断
 - 猜对，跳出循环，游戏结束
 - 猜错了，根据结果，给出提示，接着猜数字，游戏继续
 - ◆ 如果猜大了，打印 sorry，您猜大了!继续下一次循环
 - ◆ 如果猜小了，打印 sorry，您猜小了!继续下一次循环

3.3 实现代码步骤

分析完毕之后，在 main()方法中，我们一起来完成代码的编写：GuessNumber.java

```
public class GuessNumber {  
    public static void main(String[] args) {  
        //1.通过 Random 类中方法 nextInt ()，生成一个 1-100 之间的随机数  
        int randomNumber = new Random().nextInt(100);  
        System.out.println("随机数已生成！");  
        //2.输入猜的数字  
        System.out.println("----请输入您猜的数字：----");  
        Scanner sc = new Scanner(System.in);  
        int enterNumber = sc.nextInt();  
        //3.通过 while 循环，进行猜数字对错判断  
        //猜对，跳出循环，游戏结束  
        while (enterNumber != randomNumber) {  
            //猜错了，根据结果，给出提示，接着猜数字，游戏继续  
            if (enterNumber > randomNumber) {  
                //如果猜大了，打印 sorry，您猜大了!继续下一次循环  
                System.out.println("sorry，您猜大了!继续下一次循环");  
            } else {  
                //如果猜小了，打印 sorry，您猜小了!继续下一次循环  
                System.out.println("sorry，您猜小了!继续下一次循环");  
            }  
            //输入猜的数字  
            System.out.println("----请输入您猜的数字：----");  
            enterNumber = sc.nextInt();  
        }  
        System.out.println("恭喜您，答对了！");  
    }  
}
```

第4章 总结

4.1 知识点总结

- 选择结构 if

- 方式 1:

格式:

```
if(条件表达式) {  
    语句体;  
}
```

执行流程:

如果条件表达式值为 **true**, 执行语句体

如果条件表达式值为 **false**, 不执行语句体

- 方式 2:

格式:

```
if(条件表达式) {  
    语句体 1;  
} else {  
    语句体 2;  
}
```

执行流程:

如果条件表达式值为 **true**, 执行语句体 1

如果条件表达式值为 **false**, 执行语句体 2

- 方式 3:

格式:

```
if(条件表达式 1) {  
    语句体 1;  
} else if (条件表达式 2) {  
    语句体 2;  
} else if (条件表达式 n...) {  
    语句体 n...  
}  
...  
else {  
    语句体 n+1;  
}
```

执行流程:

哪一个条件表达式成立, 执行对应的语句体,

如果所有的条件表达式都不成立, 执行最后 **else** 中的语句体

- 注意事项:

- 1, 条件表达式无论是简单还是复杂, 结果都是 **boolean** 类型的值
 - 2, 如果语句体只有一条语句, 大括号可以省略,
如果是多条语句, 大括号不能省略, 建议什么时候都写上大括号
- 循环语句 For
 - 格式:
`for (初始化表达式; 条件表达式; 控制条件表达式) {`
 语句体;
`}`
 - 执行流程:
 - 1, 初始化表达式;
 - 2, 条件表达式
 如果结果为 **true**, 执行语句体
 如果结果为 **false**, 循环结束
 - 3, 当语句体执行后, 执行控制条件表达式
 - 4, 接下来, 回到第二步, 直到条件表达式为 **false** 循环结束
 - 循环语句 while
 - 格式:
 初始化表达式;
`while (条件表达式) {`
 语句体;
 控制条件表达式;
`}`
 - 执行流程:
 - 1, 初始化表达式;
 - 2, 条件表达式
 如果结果为 **true**, 执行语句体
 如果结果为 **false**, 循环结束
 - 3, 当语句体执行后, 执行控制条件表达式
 - 4, 接下来, 回到第二步, 直到条件表达式为 **false** 循环结束
 - 循环语句 do...while
 - 格式:
 初始化表达式;
`do {`
 语句体;
 控制条件表达式;
`} while (条件表达式);`
 - 执行流程:
 - 1, 初始化表达式;
 - 2, 执行语句体
 - 3, 执行控制条件表达式
 - 4, 执行条件表达式,
 如果结果为 **true**, 继续执行语句体
 如果结果为 **false**, 循环结束
 - 跳转控制语句

break; 跳出循环

continue; 结束本次循环

- Scanner 类

- 获取键盘录入的数据，对获取数据的具体操作进行了封装，只需要调用方法，即可得到键盘录入的数据。

- 常用的方法

- public int nextInt(): 获取键盘录入的数据，返回 int 类型值

- public String next(): 获取键盘录入的数据，返回 String 类型值

- Random 类：随机数类，用来产生多种类型的随机数

- 方法：

- public int nextInt(int n) 返回一个，在 0（包括）和指定值（不包括）之间 int 随机数